R Programming Basics

AGST 50104: Experimental Design

Dr. Samuel B Fernandes

2026-01-01

Table of contents

1 Introduction to R **3**

2 R as a Calculator **3**
2.1 Basic Arithmetic . . . 3
2.2 Assignment Operator . . . 4

3 Data Types in R **4**
3.1 Numeric . . . 4
3.2 Character (Strings) . . . 4
3.3 Logical (Boolean) . . . 4
3.4 Checking Data Types . . . 4

4 Vectors: Collections of Data **5**
4.1 Creating Vectors . . . 5
4.2 Vector Operations . . . 5
4.3 Useful Vector Functions . . . 5
4.4 Accessing Vector Elements . . . 6

5 Data Frames: Your Data Tables **6**
5.1 Creating Data Frames . . . 6
5.2 Accessing Data Frame Elements . . . 6
5.3 Data Frame Functions . . . 7

6 Introduction to tidyverse **7**
6.1 Core tidyverse Packages . . . 8
6.2 Installing and Loading tidyverse . . . 8

7 dplyr: Data Manipulation **8**
7.1 The Pipe Operator: `|>` . . . 8
7.2 Key dplyr Functions . . . 8
7.2.1 `filter()`: Select Rows . . . 8
7.2.2 `select()`: Choose Columns . . . 9
7.2.3 `mutate()`: Create New Columns . . . 9

7.2.4 `arrange()`: Sort Rows . . . 9
7.2.5 `summarize()`: Aggregate Data . . . 10
7.2.6 `group_by()`: Group Operations . . . 10
7.3 Combining dplyr Functions . . . 10

8 ggplot2: Data Visualization **11**
8.1 ggplot2 Syntax . . . 11
8.2 Common Geometries . . . 11
8.2.1 Bar Chart . . . 11
8.2.2 Scatter Plot . . . 11
8.2.3 Line Plot . . . 11
8.2.4 Boxplot . . . 12
8.2.5 Histogram . . . 12
8.3 Customizing Plots . . . 13
8.3.1 Colors . . . 13
8.3.2 Themes . . . 13
8.3.3 Text Size . . . 13
8.3.4 Faceting (Small Multiples) . . . 13

9 Working with Data Files **14**
9.1 Reading Data . . . 14
9.1.1 CSV Files . . . 14
9.1.2 Excel Files . . . 14
9.1.3 Text Files . . . 14
9.2 Writing Data . . . 14

10 Common R Tasks for Experimental Design **14**
10.1 Creating Randomized Designs . . . 14
10.2 Summary Statistics by Group . . . 15
10.3 Creating Publication-Quality Plots . . . 15

11 Best Practices **15**
11.1 Coding Style . . . 15
11.2 Reproducibility . . . 16
11.3 Error Prevention . . . 16

12 Troubleshooting Common Errors **16**
12.1 Object Not Found . . . 16
12.2 Incorrect Number of Dimensions . . . 16
12.3 Non-Numeric Argument . . . 16
12.4 Package Not Installed . . . 17
12.5 Syntax Errors . . . 17

13 Resources and Further Learning **17**
13.1 Official Documentation . . . 17
13.2 Free Online Books . . . 17
13.3 Community and Help . . . 17
13.4 Cheat Sheets . . . 18

14 Practice Exercises **18**
14.1 Exercise 1: Vectors and Basic Operations 18
14.2 Exercise 2: Data Frames 18
14.3 Exercise 3: Visualization 18
14.4 Exercise 4: Complete Analysis 18

15 Answers to Practice Exercises **19**
15.1 Exercise 1: Vectors and Basic Operations 19
15.2 Exercise 2: Data Frames 19
15.3 Exercise 3: Visualization 19
15.4 Exercise 4: Complete Analysis 20

1 Introduction to R

R is a programming language and environment specifically designed for statistical computing and graphics. It's free, open-source, and widely used in data science, statistics, and research.

2 R as a Calculator

R can perform basic arithmetic operations just like a calculator.

2.1 Basic Arithmetic

```
# Addition
2 + 2

# Subtraction
10 - 3

# Multiplication
4 * 5

# Division
20 / 4

# Exponentiation
2^3

# Order of operations (PEMDAS)
(5 + 3) * 2
```

2.2 Assignment Operator

In R, we use the **assignment operator** `<-` to create variables and store values.

```
# Create variables
nitrogen_rate <- 100  # kg/ha
plot_area <- 5        # m²
plots_per_treatment <- 6
total_plots <- 4 * plots_per_treatment

# View the value
total_plots
```

Alternative: You can also use `=` for assignment, but `<-` is preferred in R.

3 Data Types in R

3.1 Numeric

Numbers (integers or decimals):

```
yield <- 2500.5
plot_number <- 12
```

3.2 Character (Strings)

Text data enclosed in quotes:

```
treatment <- "Control"
variety <- "Pioneer 3751"
```

3.3 Logical (Boolean)

TRUE or FALSE values:

```
significant <- TRUE
outlier <- FALSE
```

3.4 Checking Data Types

```
class(yield)          # "numeric"
class(treatment)      # "character"
class(significant)    # "logical"
```

4 Vectors: Collections of Data

A **vector** is a collection of values of the same type. Vectors are the fundamental data structure in R.

4.1 Creating Vectors

Use the `c()` function to combine values:

```
# Numeric vector
yields <- c(2500, 2750, 2600, 2900, 2800, 2650)
yields

# Character vector
treatments <- c("Control", "N50", "N100", "N150")
treatments

# Sequence of numbers
plots <- 1:24  # Creates 1, 2, 3, ..., 24
```

4.2 Vector Operations

R performs operations element-wise on vectors:

```
# Multiply all yields by a constant
yields_tons <- yields / 1000

# Add a constant to all values
adjusted_yields <- yields + 100

# Element-wise operations between vectors
differences <- c(10, 20, 30) + c(5, 5, 5)
```

4.3 Useful Vector Functions

```
# Statistical summaries
mean(yields)          # Average
sd(yields)            # Standard deviation
median(yields)        # Median
min(yields)           # Minimum
max(yields)           # Maximum
sum(yields)           # Total sum
length(yields)        # Number of elements
```

```r
# Comprehensive summary
summary(yields)
```

4.4 Accessing Vector Elements

Use square brackets `[]` for indexing:

```r
# First element (R uses 1-based indexing!)
yields[1]

# First three elements
yields[1:3]

# Specific elements
yields[c(1, 3, 5)]

# Exclude elements (negative indexing)
yields[-1]           # All except first
yields[-c(1, 2)]     # All except first and second
```

5 Data Frames: Your Data Tables

A **data frame** is like a spreadsheet—rows are observations, columns are variables. Each column can be a different data type.

5.1 Creating Data Frames

```r
# Create a data frame manually
trial_data <- data.frame(
  treatment = c("Control", "N50", "N100", "N150"),
  yield = c(2400, 2650, 2900, 2950),
  protein = c(10.2, 11.5, 12.8, 13.1)
)

# View the data frame
trial_data
```

5.2 Accessing Data Frame Elements

```r
# Access a column by name (returns a vector)
trial_data$yield
trial_data$treatment

# Access by position [row, column]
trial_data[1, 2]      # First row, second column
trial_data[1, ]       # Entire first row
trial_data[, 2]       # Entire second column

# Access multiple rows/columns
trial_data[1:2, ]     # First two rows
trial_data[, c(1, 2)] # First two columns
```

5.3 Data Frame Functions

```r
# Structure of the data frame
str(trial_data)

# First few rows
head(trial_data)

# Last few rows
tail(trial_data)

# Dimensions
nrow(trial_data)      # Number of rows
ncol(trial_data)      # Number of columns
dim(trial_data)       # Rows and columns

# Column names
names(trial_data)
colnames(trial_data)

# Summary statistics
summary(trial_data)
```

6 Introduction to tidyverse

The **tidyverse** is a collection of R packages designed for data science. It provides a consistent, readable syntax for data manipulation and visualization.

6.1 Core tidyverse Packages

- `dplyr`: Data manipulation (filter, select, mutate, summarize)
- `ggplot2`: Data visualization
- `tidyr`: Data tidying and reshaping
- `readr`: Reading data files
- `purrr`: Functional programming tools
- `stringr`: String manipulation

6.2 Installing and Loading tidyverse

Install once (in your R console, not in scripts):

```
install.packages("tidyverse")
```

Load every session (in your scripts/Quarto documents):

```
library(tidyverse)  # Loads dplyr, ggplot2, and others
```

7 dplyr: Data Manipulation

`dplyr` provides intuitive functions for common data manipulation tasks.

7.1 The Pipe Operator: |>

The pipe operator `|>` (or `%>%`) passes the output of one function to the next. It makes code more readable.

Read it as: "Take this data, THEN do this, THEN do that…"

```
# Without pipe (nested functions - hard to read)
mean(filter(trial_data, yield > 2500)$yield)

# With pipe (sequential steps - easy to read)
trial_data |>
  filter(yield > 2500) |>
  pull(yield) |>
  mean()
```

7.2 Key dplyr Functions

7.2.1 `filter()`: Select Rows

```r
# Filter rows where yield > 2500
trial_data |>
  filter(yield > 2500)

# Multiple conditions (AND)
trial_data |>
  filter(yield > 2500 & protein > 11)

# OR condition
trial_data |>
  filter(treatment == "Control" | treatment == "N150")
```

7.2.2 `select()`: Choose Columns

```r
# Select specific columns
trial_data |>
  select(treatment, yield)

# Select by exclusion
trial_data |>
  select(-protein)

# Select columns by pattern
trial_data |>
  select(starts_with("t"))
```

7.2.3 `mutate()`: Create New Columns

```r
# Add new column
trial_data |>
  mutate(yield_tons = yield / 1000)

# Multiple new columns
trial_data |>
  mutate(
    yield_tons = yield / 1000,
    high_protein = protein > 12
  )
```

7.2.4 `arrange()`: Sort Rows

```r
# Sort by yield (ascending)
trial_data |>
  arrange(yield)

# Sort descending
trial_data |>
  arrange(desc(yield))

# Sort by multiple columns
trial_data |>
  arrange(protein, yield)
```

7.2.5 `summarize()`: Aggregate Data

```r
# Calculate summary statistics
trial_data |>
  summarize(
    mean_yield = mean(yield),
    sd_yield = sd(yield),
    max_yield = max(yield)
  )
```

7.2.6 `group_by()`: Group Operations

```r
# Group by treatment, then summarize within groups
trial_data |>
  group_by(treatment) |>
  summarize(
    mean_yield = mean(yield),
    n = n()  # Count observations
  )
```

7.3 Combining dplyr Functions

The power of dplyr comes from chaining operations:

```r
trial_data |>
  filter(yield > 2500) |>
  mutate(yield_tons = yield / 1000) |>
  select(treatment, yield_tons) |>
  arrange(desc(yield_tons))
```

8 ggplot2: Data Visualization

`ggplot2` creates publication-quality graphics using a **grammar of graphics**—you build plots in layers.

8.1 ggplot2 Syntax

```
ggplot(data = <DATA>, aes(x = <X>, y = <Y>)) +  # 1. Data + aesthetics
  geom_<TYPE>() +                               # 2. Geometry (plot type)
  labs(title = "Title", x = "X label", y = "Y label") +  # 3. Labels
  theme_<THEME>()                               # 4. Theme
```

8.2 Common Geometries

8.2.1 Bar Chart

```
ggplot(trial_data, aes(x = treatment, y = yield)) +
  geom_col(fill = "#2E8B57", alpha = 0.8) +
  labs(
    title = "Wheat Yield by Nitrogen Treatment",
    x = "Nitrogen Treatment",
    y = "Yield (kg/ha)"
  ) +
  theme_minimal()
```

8.2.2 Scatter Plot

```
ggplot(trial_data, aes(x = protein, y = yield)) +
  geom_point(size = 3, color = "#4A90E2") +
  labs(
    title = "Yield vs. Protein Content",
    x = "Protein (%)",
    y = "Yield (kg/ha)"
  ) +
  theme_minimal()
```

8.2.3 Line Plot

```
ggplot(trial_data, aes(x = 1:4, y = yield)) +
  geom_line(color = "#FF8C42", linewidth = 1.5) +
  geom_point(size = 3) +
  labs(
    title = "Yield Trend Across Treatments",
    x = "Treatment Order",
    y = "Yield (kg/ha)"
  ) +
  theme_minimal()
```

8.2.4 Boxplot

```
# For boxplots, you typically need grouped data
expanded_data <- data.frame(
  treatment = rep(c("Control", "N50", "N100", "N150"), each = 6),
  yield = c(
    rnorm(6, 2400, 100),
    rnorm(6, 2650, 120),
    rnorm(6, 2900, 110),
    rnorm(6, 2950, 130)
  )
)

ggplot(expanded_data, aes(x = treatment, y = yield, fill = treatment)) +
  geom_boxplot(alpha = 0.7) +
  scale_fill_manual(values = c("#9D2235", "#FF8C42", "#2E8B57", "#4A90E2")) +
  labs(
    title = "Yield Distribution by Treatment",
    x = "Treatment",
    y = "Yield (kg/ha)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

8.2.5 Histogram

```
ggplot(expanded_data, aes(x = yield)) +
  geom_histogram(bins = 15, fill = "#2E8B57", color = "white") +
  labs(
    title = "Distribution of Yield Values",
    x = "Yield (kg/ha)",
    y = "Frequency"
  ) +
  theme_minimal()
```

8.3 Customizing Plots

8.3.1 Colors

```
# Manual colors
ggplot(trial_data, aes(x = treatment, y = yield, fill = treatment)) +
  geom_col() +
  scale_fill_manual(values = c("#9D2235", "#FF8C42", "#2E8B57", "#4A90E2"))

# Built-in palettes
ggplot(trial_data, aes(x = treatment, y = yield, fill = treatment)) +
  geom_col() +
  scale_fill_brewer(palette = "Set2")
```

8.3.2 Themes

```
# Built-in themes
theme_minimal()      # Clean, minimal
theme_classic()      # Classic look with axes
theme_bw()           # Black and white
theme_dark()         # Dark background
theme_void()         # Completely blank
```

8.3.3 Text Size

```
ggplot(trial_data, aes(x = treatment, y = yield)) +
  geom_col() +
  theme_minimal(base_size = 16) +  # Base font size
  theme(
    plot.title = element_text(size = 20, face = "bold"),
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12)
  )
```

8.3.4 Faceting (Small Multiples)

```
# Split plot by a categorical variable
ggplot(expanded_data, aes(x = yield)) +
  geom_histogram(bins = 10, fill = "#2E8B57") +
  facet_wrap(~treatment, ncol = 2) +
  theme_minimal()
```

9 Working with Data Files

9.1 Reading Data

9.1.1 CSV Files

```r
# Base R
data <- read.csv("mydata.csv")

# tidyverse (readr) - better defaults
library(readr)
data <- read_csv("mydata.csv")
```

9.1.2 Excel Files

```r
library(readxl)
data <- read_excel("mydata.xlsx", sheet = "Sheet1")
```

9.1.3 Text Files

```r
# Tab-delimited
data <- read.table("mydata.txt", header = TRUE, sep = "\t")

# Space-delimited
data <- read.table("mydata.txt", header = TRUE)
```

9.2 Writing Data

```r
# CSV
write.csv(data, "output.csv", row.names = FALSE)

# tidyverse version
write_csv(data, "output.csv")
```

10 Common R Tasks for Experimental Design

10.1 Creating Randomized Designs

```r
# Completely Randomized Design (CRD)
set.seed(2026)  # For reproducibility
treatments <- rep(c("Control", "Low", "Medium", "High"), each = 6)
randomized <- sample(treatments)
crd <- data.frame(plot = 1:24, treatment = randomized)
```

10.2 Summary Statistics by Group

```r
trial_data |>
  group_by(treatment) |>
  summarize(
    n = n(),
    mean = mean(yield),
    sd = sd(yield),
    se = sd(yield) / sqrt(n())
  )
```

10.3 Creating Publication-Quality Plots

```r
ggplot(trial_data, aes(x = treatment, y = yield)) +
  geom_col(fill = "#2E8B57", alpha = 0.8) +
  geom_text(aes(label = yield), vjust = -0.5, size = 4) +
  labs(
    title = "Wheat Yield by Nitrogen Treatment",
    subtitle = "2026 Field Trial",
    x = "Nitrogen Rate (kg/ha)",
    y = "Yield (kg/ha)",
    caption = "Error bars represent ± 1 SE"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5),
    panel.grid.major.x = element_blank()
  )
```

11 Best Practices

11.1 Coding Style

- **Use meaningful variable names**: `nitrogen_rate` not `x`

- **Use snake_case** for names: `trial_data` not `TrialData`
- **Add comments** to explain complex logic
- **Keep lines short** (< 80 characters when possible)
- **Use spaces** around operators: `x <- 5` not `x<-5`
- **Indent code** inside functions and loops

11.2 Reproducibility

- **Always use `set.seed()`** before random operations
- **Document package versions** if critical
- **Use relative paths** for file locations
- **Avoid hard-coded values**—use variables
- **Comment your code** to explain why, not just what

11.3 Error Prevention

- **Test code incrementally**—run line by line
- **Check data types** with `class()` and `str()`
- **Use `head()` and `summary()`** to inspect data
- **Read error messages carefully**—they're often helpful
- **Use descriptive variable names** to avoid confusion

12 Troubleshooting Common Errors

12.1 Object Not Found

```
yield  # Error: object 'yield' not found
```

Solution: The variable hasn't been created yet. Define it first.

12.2 Incorrect Number of Dimensions

```
trial_data[1]  # Returns a data frame, not a value
```

Solution: Use `trial_data$yield[1]` or `trial_data[1, 2]` to get a specific value.

12.3 Non-Numeric Argument

```
mean(treatments)  # Error: argument is not numeric
```

Solution: You can only calculate `mean()` on numeric vectors.

12.4 Package Not Installed

```
library(ggplot2)  # Error: package not installed
```

Solution: Install it first: `install.packages("ggplot2")`

12.5 Syntax Errors

```
data <- c(1, 2, 3  # Missing closing parenthesis
```

Solution: Check for matching parentheses, brackets, and quotes.

13 Resources and Further Learning

13.1 Official Documentation

- **R Project:** r-project.org
- **CRAN Task Views:** cran.r-project.org/web/views
- **tidyverse:** tidyverse.org
- **ggplot2:** ggplot2.tidyverse.org
- **Video Playlist (intro R):** YouTube: R Programming Basics for Data Science

13.2 Free Online Books

- **R for Data Science (2e):** r4ds.hadley.nz
- **ggplot2 Book:** ggplot2-book.org
- **Advanced R:** adv-r.hadley.nz
- **R Graphics Cookbook:** r-graphics.org

13.3 Community and Help

- **Stack Overflow:** stackoverflow.com/questions/tagged/r
- **RStudio Community:** community.rstudio.com
- **R-bloggers:** r-bloggers.com
- **Twitter/X:** Follow #rstats hashtag

13.4 Cheat Sheets

RStudio provides excellent cheat sheets:

- **Base R Cheat Sheet**
- **dplyr Cheat Sheet**
- **ggplot2 Cheat Sheet**
- **RStudio IDE Cheat Sheet**

Download at: rstudio.com/resources/cheatsheets

14 Practice Exercises

14.1 Exercise 1: Vectors and Basic Operations

1. Create a vector of 6 yield values
2. Calculate the mean, standard deviation, and range
3. Create a new vector with yields increased by 10%
4. Find which yields are above the mean

14.2 Exercise 2: Data Frames

1. Create a data frame with treatment, yield, and protein columns
2. Add a new column for yield in tons (divide by 1000)
3. Filter to show only treatments with yield > 2600
4. Calculate the mean yield by treatment

14.3 Exercise 3: Visualization

1. Create a bar chart of yield by treatment
2. Customize the colors and add labels
3. Change the theme to `theme_classic()`
4. Add a title and axis labels

14.4 Exercise 4: Complete Analysis

1. Create a randomized experimental design
2. Simulate yield data for each plot
3. Calculate summary statistics by treatment
4. Create a boxplot of yields by treatment
5. Save the plot as a PNG file

For questions or help, contact Dr. Samuel B Fernandes or visit office hours.

15 Answers to Practice Exercises

15.1 Exercise 1: Vectors and Basic Operations

```
# Yield vector (kg/ha)
yields <- c(2500, 2650, 2600, 2800, 2550, 2900)

# Summary statistics
mean_yield <- mean(yields)
sd_yield <- sd(yields)
range_yield <- range(yields)  # min and max

# Increase yields by 10%
yields_plus10 <- yields * 1.10

# Values above the mean
above_mean <- yields[yields > mean_yield]
```

15.2 Exercise 2: Data Frames

```
library(dplyr)

trial_data <- data.frame(
  treatment = c("Control", "N50", "N100", "N150"),
  yield = c(2400, 2650, 2900, 2950),
  protein = c(10.2, 11.5, 12.8, 13.1)
)

# Yield in tons
trial_data <- trial_data |> mutate(yield_tons = yield / 1000)

# Filter yield > 2600
high_yield <- trial_data |> filter(yield > 2600)

# Mean yield by treatment
mean_by_trt <- trial_data |>
  group_by(treatment) |>
  summarize(mean_yield = mean(yield), .groups = "drop")
```

15.3 Exercise 3: Visualization

```r
library(ggplot2)

ggplot(trial_data, aes(x = treatment, y = yield, fill = treatment)) +
  geom_col(alpha = 0.8) +
  geom_text(aes(label = yield), vjust = -0.5) +
  scale_fill_manual(values = c("#9D2235", "#FF8C42", "#2E8B57", "#4A90E2")) +
  labs(
    title = "Yield by Treatment",
    x = "Treatment",
    y = "Yield (kg/ha)"
  ) +
  theme_classic()
```

15.4 Exercise 4: Complete Analysis

```r
library(dplyr)
library(ggplot2)

set.seed(2026)

# 1. Randomized design
treatments <- rep(c("Control", "Low", "Medium", "High"), each = 6)
crd <- data.frame(
  plot = 1:24,
  treatment = sample(treatments)
)

# 2. Simulate yield data
yield_means <- c(Control = 2400, Low = 2600, Medium = 2850, High = 3000)
crd <- crd |>
  mutate(yield = rnorm(n(), mean = yield_means[treatment], sd = 120))

# 3. Summary by treatment
summary_stats <- crd |>
  group_by(treatment) |>
  summarize(
    n = n(),
    mean = mean(yield),
    sd = sd(yield),
    se = sd / sqrt(n()),
    .groups = "drop"
  )

# 4. Boxplot
plot_yield <- ggplot(crd, aes(x = treatment, y = yield, fill = treatment)) +
```

```r
  geom_boxplot(alpha = 0.7) +
  scale_fill_manual(values = c("#9D2235", "#FF8C42", "#2E8B57", "#4A90E2")) +
  labs(
    title = "Simulated Yield by Treatment",
    x = "Treatment",
    y = "Yield (kg/ha)"
  ) +
  theme_minimal()

# 5. Save plot (optional)
ggsave("yield_boxplot.png", plot_yield, width = 7, height = 5, dpi = 120)
```

End of R Basics Introduction